LangChain & LangGraph Study Guide

From basic LLM calls to production-grade multi-agent systems

Free ebook source: github.com/rahulkolekardev/langchain-langgraph-agentic-ebook

Phase 0 - Prerequisites & Setup

Before you build sophisticated agents and retrieval-augmented generation (RAG) systems, you need a solid **Python foundation**, a working **development environment**, and at least one **LLM provider** configured.

This phase ensures you can:

- Write and run basic Python scripts (sync and async).
- Create isolated project environments with `venv` and `pip`.
- Install and configure LangChain, LangGraph, and dependencies.
- Call an LLM (e.g. OpenAI) from Python.

1. Core Python Skills

LangChain and LangGraph are Python libraries. You don't need to be a "Python expert", but you should be comfortable with:

- Functions and classes
- Type hints
- Basic async/await
- Project layout and virtual environments

1.1 Functions and Type Hints

Functions let you encapsulate logic. Type hints make your code easier to understand and catch mistakes early with tools like `mypy` or your IDE.

```
from typing import List

def summarize_points(points: List[str]) -> str:
    """Return a short summary of a list of bullet points."""
    joined = "; ".join(points)
    return f"In summary, we covered: {joined}."

if __name__ == "__main__":
    topics = ["LangChain basics", "LangGraph workflows", "RAG"]
    print(summarize_points(topics))
```

1.2 Classes

Many LangChain components are classes (models, prompts, chains). Understanding how classes work helps you customize and wrap them.

```
class SimpleCounter:
    def __init__(self) -> None:
        self.value = 0

    def increment(self, step: int = 1) -> None:
        self.value += step

    def reset(self) -> None:
        self.value = 0

if __name__ == "__main__":
    counter = SimpleCounter()
    counter.increment()
    counter.increment(5)
    print("Counter value:", counter.value)
```

1.3 Virtual Environments & Project Structure

A virtual environment isolates your project's dependencies so they don't conflict with other Python projects.

```
# 1. Create a folder for this course
mkdir langchain-langgraph-study
cd langchain-langgraph-study

# 2. Create a virtual environment (Linux/macOS)
python -m venv .venv
source .venv/bin/activate

# On Windows PowerShell:
# python -m venv .venv
# .venv\Scripts\Activate.ps1

# 3. Upgrade pip
python -m pip install --upgrade pip
```

A simple layout for this ebook could be:

```
langchain-langgraph-study/
   .venv/
   src/
      phase0/
      phase1/
      ...
   Readme.md
   requirements.txt
```

1.4 Basic AsyncIO

LangChain and LangGraph both support async execution. Knowing how to use `async def` and `await` lets you:

- Call models in parallel.
- Build responsive APIs.

```
import asyncio

async def fetch_answer(question: str) -> str:
    # In real code, you'll call an LLM here.
    await asyncio.sleep(0.5)
    return f"Mock answer to: {question}"

async def main() -> None:
    questions = ["What is LangChain?", "What is LangGraph?"]
    tasks = [fetch_answer(q) for q in questions]
    answers = await asyncio.gather(*tasks)
    for q, a in zip(questions, answers):
        print(q, "->", a)

if __name__ == "__main__":
    asyncio.run(main())
```

2. LLM Basics

Large Language Models (LLMs) are the core engine behind LangChain applications. LangChain provides a unified interface to different providers (OpenAI, Anthropic, local models such as Ollama, and more).

2.1 Key Concepts

- **Prompt** - the text you send to the model.
- **Context window** - maximum tokens (input + output) the model can process.
- **Tokens** - chunks of text; pricing and limits are per token.
- **Temperature** - randomness; low = deterministic, high = creative.
- **Chat vs. completion models** - structured messages vs.

raw text.

2.2 Chat vs. Text Completion

Modern APIs (like OpenAI's `gpt-4o`) use a **chat-style** interface, where you send a list of messages with roles like `system`, `user`, and `assistant`.

```
# Pseudo-example using OpenAI's chat completion style (without LangChain)
from openai import OpenAI
import os

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a concise assistant."},
        {"role": "user", "content": "Explain LangChain in one paragraph."},
    ],
    temperature=0.2,
)

print(response.choices[0].message.content)
```

LangChain wraps this in a higher-level interface that integrates with prompts, tools, and document workflows. You will see that starting in Phase 1.

3. Environment Setup

Next, you'll set up a project that can run both LangChain and LangGraph.

3.1 Install Dependencies

```
# Inside your activated virtual environment
pip install \
  langchain langchain-core langchain-community \
  langchain-openai langgraph python-dotenv \
  faiss-cpu chromadb
```

This installs:

- `langchain` and `langchain-core` - core abstractions and LCEL.
- `langchain-openai` - OpenAI chat and embedding wrappers.
- `langchain-community` - community integrations (vector stores, loaders).
- `langgraph` - orchestration framework for stateful workflows.
- `python-dotenv` - load API keys from `.env`.
- `faiss-cpu` / `chromadb` - common vector stores for RAG.

3.2 Store Secrets in `.env`

Never hard-code API keys directly in your source code. Instead, store them in a `.env` file and load them at runtime.

```
# .env (do not commit to Git)
OPENAI_API_KEY=sk-your-openai-key-here
ANTHROPIC_API_KEY=your-anthropic-key-optional
```

```
# src/phase0/load_env_example.py
import os
from dotenv import load_dotenv

def load_env() -> None:
    load_dotenv()
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
```

```
        raise RuntimeError("OPENAI_API_KEY is not set in your .env file")
    print("API key loaded successfully (hidden).")

if __name__ == "__main__":
    load_env()
```

3.3 Verifying the Installation

Create a quick script to ensure LangChain and LangGraph import correctly.

```
# src/phase0/verify_install.py
from langchain_core.runnables import RunnableLambda
from langgraph.graph import StateGraph

def say_hello(name: str) -> str:
    return f"Hello, {name}!"

def main() -> None:
    # Test LangChain LCEL
    chain = RunnableLambda(say_hello)
    print(chain.invoke("LangChain"))

    # Test LangGraph basic graph
    class State(dict):
        pass

    def greet(state: State) -> State:
        name = state.get("name", "world")
        return {"greeting": f"Hello from LangGraph, {name}!"}

    graph = StateGraph(State)
    graph.add_node("greet", greet)
    graph.set_entry_point("greet")
    app = graph.compile()
    result = app.invoke({"name": "LangGraph"})
    print(result["greeting"])

if __name__ == "__main__":
    main()
```

4. Reading Docs Strategically

LangChain and LangGraph have extensive documentation. At this point, you don't need to read everything in depth. Instead:

- Skim the main sections (models, prompts, chains, agents, document loaders).
- Skim LangGraph's concepts (graphs, nodes, state, edges).
- Note where examples live so you can refer back later.
- Briefly look at observability tools like **LangSmith** (or similar tracing systems) so you know that detailed traces and evaluations are available once you reach the production and evaluation phases.

Goal: know what exists and roughly where to find it, not memorize every API.

Mini Task - First LangChain Script

To finish Phase 0, you'll write a small script that:

1. Loads your API key from `.env`.
2. Asks the user for a question.
3. Calls a chat model using LangChain.

4. Prints the answer.

5.1 Basic Chat Script with LangChain

```
# src/phase0/first_langchain_chat.py
import os
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage


def main() -> None:
    load_dotenv()

    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise RuntimeError("Set OPENAI_API_KEY in your .env file first.")

    # Configure a chat model (you can change model name)
    model = ChatOpenAI(
        model="gpt-4o-mini",
        temperature=0.2,
    )

    print("Ask a question about LangChain or LangGraph:")
    question = input("> ").strip()
    if not question:
        print("No question provided, exiting.")
        return

    messages = [
        SystemMessage(content="You are a helpful assistant for LangChain and LangGraph."),
        HumanMessage(content=question),
    ]

    response = model.invoke(messages)
    print("\n--- Answer ---")
    print(response.content)


if __name__ == "__main__":
    main()
```

5.2 Running the Script

```
# From your project root
export OPENAI_API_KEY="sk-..."  # or configure in .env and just run
python src/phase0/first_langchain_chat.py
```

If everything is wired correctly, you'll see a response from the model. You've now completed the minimum prerequisites to start building with LangChain and LangGraph.

Phase 1 - LangChain Core & LCEL

In Phase 1, you will learn how LangChain structures LLM applications and how to use the **LangChain Expression Language (LCEL)** to compose reusable chains.

By the end of this phase you will be able to:

- Build prompt-driven chains using LCEL.
- Load and split documents, and perform basic RAG.
- Create tools, simple agents, and memory-backed chatbots.

Module 1 - LangChain Core Concepts & LCEL

This module introduces the main building blocks in LangChain and how LCEL lets you connect them into pipelines using a simple, composable syntax.

1.1 What LangChain Gives You

LangChain sits *above* your model provider and provides a toolkit for building applications:

- **Model abstractions** - unify different providers (OpenAI, etc.).
- **Prompts** - structured templates for messages.
- **Output parsers** - safely parse raw LLM output into structured types.
- **Document loaders & retrievers** - ingest data and perform retrieval.
- **Tools & agents** - let models call functions and make decisions.

Most of these pieces are implemented as **Runnables**, which can be composed with LCEL.

1.2 Runnables & the Pipe Operator

LCEL is the "glue" of LangChain. A **Runnable** is any component that:

- Takes an input.
- Returns an output.
- Supports standard methods like `.invoke`, `.batch`, and `.stream`.

You connect runnables using the `|` ("pipe") operator:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a concise assistant."),
        ("human", "Explain {topic} in 3 bullet points."),
    ]
)

model = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
parser = StrOutputParser()
```

```
chain = prompt | model | parser

result = chain.invoke({"topic": "LangChain"})
print(result)
```

Here, the chain consists of three runnables:

1. `prompt` - turns input dict into messages.
2. `model` - calls the LLM with those messages.
3. `parser` - converts the model output into a string.

1.3 Core Methods: `.invoke`, `.batch`, `.stream`, `.astream`

LCEL chains support a standard set of methods:

- `.invoke(input)` - run once with a single input.
- `.batch(inputs)` - run on a list of inputs (sync).
- `.stream(input)` - generator yielding partial outputs.
- `.astream(input)` - async version of stream.

```
topics = ["RAG", "LangGraph", "LLM evaluation"]

# 1) Batch
for output in chain.batch([{"topic": t} for t in topics]):
    print("---")
    print(output)

# 2) Streaming
print("\nStreaming explanation of LangGraph:\n")
for chunk in chain.stream({"topic": "LangGraph"}):
    # Each chunk is a partial string
    print(chunk, end="", flush=True)
```

1.4 Advanced Building Blocks: RunnableLambda, RunnableParallel, RunnableBranch

LCEL includes helpers that let you inject custom Python logic or branch/merge flows.

```
from langchain_core.runnables import RunnableLambda, RunnableParallel, RunnableBranch

def to_uppercase(text: str) -> str:
    return text.upper()

uppercase = RunnableLambda(to_uppercase)

# Parallel: run multiple chains and collect results as a dict
parallel = RunnableParallel(
    original=lambda x: x,
    shouting=uppercase,
)

print(parallel.invoke("hello LCEL"))
# {'original': 'hello LCEL', 'shouting': 'HELLO LCEL'}
```

`RunnableBranch` lets you choose a path based on the input:

```
def is_short(input: dict) -> bool:
    return len(input["text"]) < 40

short_chain = RunnableLambda(lambda d: f"Short text: {d['text']}")
long_chain = RunnableLambda(lambda d: f"Long text: {d['text']}")

branch = RunnableBranch(
    (is_short, short_chain),
    (lambda d: True, long_chain),  # default branch
)

print(branch.invoke({"text": "Hi"}))
print(branch.invoke({"text": "This is a long piece of text for demonstration."}))
```

1.5 Configuration: `.with_config`, Retries & Fallbacks

LCEL supports attaching configuration metadata (for tracing, tagging, etc.) and wrapping chains with retries or fallbacks.

```
configured_chain = chain.with_config(
    tags=["phase1", "module1"],
    metadata={"purpose": "educational-example"},
)

# Using a simple fallback: try a more capable model if the first fails
fallback_model = ChatOpenAI(model="gpt-4o", temperature=0.2)
fallback_chain = prompt | fallback_model | parser

robust_chain = configured_chain.with_fallbacks([fallback_chain])

print(robust_chain.invoke({"topic": "LCEL configuration"}))
```

1.6 Simple Prompt Chaining

Now you can build a simple reusable chain that:

1. Takes a topic as input.
2. Generates an explanation + bullet points.
3. Supports normal, batch, and streaming use.

```
def build_topic_explainer():
    from langchain_core.prompts import ChatPromptTemplate
    from langchain_openai import ChatOpenAI
    from langchain_core.output_parsers import StrOutputParser

    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a senior AI engineer and educator."),
            (
                "human",
                "Explain the topic '{topic}' in a short paragraph "
                "followed by 3 bullet points.",
            ),
        ]
    )

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
    parser = StrOutputParser()
    return prompt | model | parser


if __name__ == "__main__":
    chain = build_topic_explainer()
    print(chain.invoke({"topic": "LangChain Expression Language (LCEL)"}))
```

Module 2 - Working with Data: Loaders, Splitters, Vector Stores & Simple RAG

This module focuses on getting external data into your LangChain app and building a basic Retrieval-Augmented Generation (RAG) system.

2.1 Document Loaders

Document loaders turn raw files or remote sources into a list of `Document` objects with `page_content` and `metadata`.

```
from langchain_community.document_loaders import TextLoader, PyPDFLoader

# Load a plain text or markdown file
text_loader = TextLoader("notes/langchain_intro.md")
text_docs = text_loader.load()

# Load a PDF
pdf_loader = PyPDFLoader("notes/langchain_paper.pdf")
pdf_docs = pdf_loader.load()
```

```
print("Text docs:", len(text_docs), "PDF docs:", len(pdf_docs))
```

Common loaders include:

- `TextLoader` - local text/markdown files.
- `PyPDFLoader` - PDFs.
- Web loaders - load HTML content from URLs.

2.2 Text Splitters

Most models can't handle large documents in a single prompt. You use text splitters to break documents into overlapping chunks.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

all_docs = text_docs + pdf_docs

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
)

chunks = splitter.split_documents(all_docs)
print("Total chunks:", len(chunks))
print("Sample chunk content:\n", chunks[0].page_content[:200])
```

2.3 Embeddings & Vector Stores

To perform semantic search, you convert text chunks into vector embeddings and store them in a vector store such as FAISS or Chroma.

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

vectorstore = FAISS.from_documents(chunks, embedding=embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

The `retriever` object is what you'll plug into your RAG chain.

2.4 Building a Simple RAG Chain with LCEL

A minimal RAG pipeline:

1. User asks a question.
2. Retriever fetches relevant chunks.
3. Prompt composes the question and context.
4. Model generates an answer.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

RAG_PROMPT = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "You are a helpful assistant. Use the provided context snippets "
            "to answer the question. If the answer is not in the context, say "
            "you don't know.",
        ),
        (
            "human",
            "Context:\n{context}\n\nQuestion: {question}\n\nAnswer step by step:",
        ),
    ]
)

def format_docs(docs):
```

```
    return "\n\n".join(d.page_content for d in docs)

retrieval_chain = (
    {"context": retriever | RunnableLambda(format_docs), "question": RunnableLambda(lambda
x: x["question"])}
    | RAG_PROMPT
    | ChatOpenAI(model="gpt-4o-mini", temperature=0.1)
    | StrOutputParser()
)
```

2.5 CLI RAG App

Wrap the chain in a simple CLI for interactive querying:

```
# src/phase1/simple_rag_cli.py
import os
from dotenv import load_dotenv

from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter


def build_rag_chain():
    loader = TextLoader("notes/course_notes.md")
    docs = loader.load()

    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
    chunks = splitter.split_documents(docs)

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = FAISS.from_documents(chunks, embedding=embeddings)
    retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You answer questions based on the given course notes. "
                "If you are not sure, say you don't know.",
            ),
            (
                "human",
                "Context:\n{context}\n\nQuestion: {question}",
            ),
        ]
    )

    def format_docs(docs):
        return "\n\n".join(d.page_content for d in docs)

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)
    parser = StrOutputParser()

    chain = (
        {
            "context": retriever | RunnableLambda(format_docs),
            "question": RunnableLambda(lambda x: x["question"]),
        }
        | prompt
        | model
        | parser
    )
    return chain


def main():
    load_dotenv()
    chain = build_rag_chain()

    print("RAG CLI over your course notes. Type 'exit' to quit.")
    while True:
        question = input("\nQuestion: ").strip()
        if not question or question.lower() in {"exit", "quit"}:
            break
```

```
        answer = chain.invoke({"question": question})
        print("\nAnswer:\n", answer)


if __name__ == "__main__":
    main()
```

Module 3 - Tools, Agents, Memory & Structured Output

This module introduces tool-calling, basic agents, conversational memory, and structured outputs.

3.1 Tools in LangChain

A **tool** is a Python function (or wrapper) that the model can invoke to perform an action: search, calculations, HTTP calls, etc.

```
from langchain_core.tools import tool

@tool
def add_numbers(x: float, y: float) -> float:
    """Add two numbers and return the result."""
    return x + y

@tool
def simple_search(query: str) -> str:
    """
    A placeholder search tool.
    In production, you would call a search API or a RAG chain here.
    """
    # e.g. call a web search API, or your own RAG pipeline:
    # return rag_chain.invoke({"question": query})
    return f"Pretend search results for: {query}"

print(add_numbers.invoke({"x": 2, "y": 3}))      # 5.0
print(simple_search.invoke({"query": "LangChain"}))
```

Tools typically define an input schema (via type hints or pydantic models) so that the LLM can call them correctly.

3.2 Simple Tool-Calling Agent

LangChain provides agent constructors that let a model decide which tool to call. Here is a simple example using an OpenAI model with tool calling:

```
# src/phase1/tool_agent.py
import os
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate


@tool
def calculator(expression: str) -> str:
    """Evaluate a basic Python arithmetic expression like '2 + 3 * 4'."""
    try:
        # WARNING: eval is dangerous in production; use a safe parser instead.
        result = eval(expression, {"__builtins__": {}}, {})
        return str(result)
    except Exception as e:
        return f"Error: {e}"


def build_agent():
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    tools = [calculator, simple_search]
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a helpful math assistant. Use tools when needed."),
            ("human", "{input}"),
            ("assistant", "Think step by step, and use the calculator tool for
arithmetic."),
        ]
```

```
    )

    agent = create_tool_calling_agent(llm, tools, prompt)
    executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
    return executor


if __name__ == "__main__":
    agent = build_agent()
    result = agent.invoke({"input": "What is 12 * (7 + 5)?"})
    print("Final answer:", result["output"])
```

3.3 Memory - Remembering Conversation History

Memory lets your chatbot remember earlier turns. The simplest form is a conversation buffer.

```
# src/phase1/chat_with_memory.py
import os
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain.chains import ConversationChain
from langchain.chains.conversation.memory import ConversationBufferMemory


def main():
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)

    memory = ConversationBufferMemory()
    chain = ConversationChain(llm=llm, memory=memory, verbose=True)

    print("Chat with memory. Type 'exit' to quit.")
    while True:
        user_input = input("\nYou: ").strip()
        if user_input.lower() in {"exit", "quit"}:
            break
        response = chain.invoke({"input": user_input})
        print("Bot:", response["response"])


if __name__ == "__main__":
    main()
```

Behind the scenes, `ConversationBufferMemory` stores all previous messages and injects them into the prompt on each turn.

3.4 Structured Outputs

Often you don't just want free-form text; you want the model to return a structured object that your code can easily consume.

```
# src/phase1/structured_output.py
from typing import List
from pydantic import BaseModel, Field
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI


class QuestionAnswer(BaseModel):
    question: str = Field(..., description="The user question.")
    answer: str = Field(..., description="The answer to the question.")
    tags: List[str] = Field(
        default_factory=list,
        description="A few short tags summarizing the topic.",
    )


def main():
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

    structured_llm = llm.with_structured_output(QuestionAnswer)

    qa = structured_llm.invoke(
        "Explain what LangChain is and give 2-3 short tags."
    )
```

```
    print("Question:", qa.question)
    print("Answer:", qa.answer)
    print("Tags:", qa.tags)


if __name__ == "__main__":
    main()
```

`with_structured_output` instructs the model (via a system prompt and tools/function-calling under the hood) to return a JSON object that matches the Pydantic schema.

3.5 Putting It Together: A Console Chatbot with Memory & a Tool

Finally, build a small console chatbot that:

- Maintains conversation history.
- Uses a calculator tool for math questions.
- Returns a structured Python object per turn.

```
# src/phase1/final_console_bot.py
import os
from typing import List

from dotenv import load_dotenv
from pydantic import BaseModel, Field

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains.conversation.memory import ConversationBufferMemory


@tool
def calculator(expression: str) -> str:
    """Safely evaluate simple arithmetic like '2 + 2 * 3'."""
    try:
        result = eval(expression, {"__builtins__": {}}, {})
        return str(result)
    except Exception as e:
        return f"Error: {e}"


class BotTurn(BaseModel):
    answer: str = Field(..., description="The main answer to the user.")
    reasoning: str = Field(..., description="Short explanation of how the answer was
derived.")
    used_calculator: bool = Field(..., description="Whether the calculator tool was
used.")


def build_agent_with_memory():
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

    tools = [calculator]
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a thoughtful assistant. Use the calculator tool for any
arithmetic.",
            ),
            ("human", "{input}"),
        ]
    )

    agent = create_tool_calling_agent(llm, tools, prompt)
    executor = AgentExecutor(agent=agent, tools=tools, verbose=False)

    # Wrap LLM for structured output
    structured_llm = llm.with_structured_output(BotTurn)
    return executor, structured_llm


def main():
```

```
    executor, structured_llm = build_agent_with_memory()

    print("Chatbot with calculator tool and structured output. Type 'exit' to quit.")
    while True:
        user_input = input("\nYou: ").strip()
        if user_input.lower() in {"exit", "quit"}:
            break

        # First let the agent run (may use tools)
        result = executor.invoke({"input": user_input})
        text_output = result["output"]

        # Then post-process via structured model
        turn = structured_llm.invoke(
            f"User said: {user_input}\n\nAssistant raw answer: {text_output}\n\n"
            "Rewrite the answer if needed and provide reasoning. "
            "Mark used_calculator=True if the answer involves numeric computation."
        )

        print("Assistant:", turn.answer)
        print("Reasoning:", turn.reasoning)
        print("Used calculator:", turn.used_calculator)


if __name__ == "__main__":
    main()
```

Phase 2 - LangGraph Fundamentals

LangGraph is a framework for building **stateful**, **long-running**, and **controlled** LLM applications using graphs. You define nodes (functions), edges (transitions), and a shared state that flows through the graph.

In this phase, you will:

- Learn core LangGraph concepts: nodes, edges, state, compilation.
- Wrap a RAG workflow from Phase 1 inside a LangGraph.
- Use branching, loops, human-in-the-loop, and persistence.

Module 4 - LangGraph Basics: Nodes, Edges & State

This module introduces the core abstractions in LangGraph and walks through building a simple, but real, workflow graph.

4.1 Why LangGraph?

LCEL is great for straightforward pipelines. But as your application grows, you may need:

- Complex branching and looping logic.
- Long-running workflows with checkpoints.
- Multiple agents coordinating over shared state.

LangGraph is designed for this kind of orchestration. You define a graph of steps rather than a single chain.

4.2 Core Concepts

- **State** - a typed container (usually a dict or `TypedDict`) passed between nodes.
- **Nodes** - Python callables that take state and return an updated state.
- **Edges** - directed connections between nodes; can be conditional.
- **Graph compilation** - you define a `StateGraph` and compile it into an app.

4.3 Defining State

State defines what your workflow carries between steps. Using `TypedDict` or Pydantic makes this explicit:

```
# src/phase2/basic_graph.py
from typing import TypedDict, List

class RAGState(TypedDict, total=False):
    question: str
    rewritten_question: str
    retrieved_docs: List[str]
    answer: str
    error: str
```

4.4 Creating Nodes

Each node is a function that:

1. Accepts a `RAGState` instance.
2. Returns a *partial* update to `RAGState`.

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import TextLoader


def build_rag_components():
    loader = TextLoader("notes/course_notes.md")
    docs = loader.load()

    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
    chunks = splitter.split_documents(docs)

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = FAISS.from_documents(chunks, embedding=embeddings)
    retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

    return retriever, llm

from typing import List
from langgraph.graph import StateGraph

retriever, llm = build_rag_components()


def start(state: RAGState) -> RAGState:
    if "question" not in state or not state["question"].strip():
        return {"error": "Question is required."}
    return {}  # no change, we just validate


def rewrite_query(state: RAGState) -> RAGState:
    question = state["question"]
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "Rewrite the user question to be clear and concise."),
            ("human", "{question}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    rewritten = chain.invoke({"question": question})
    return {"rewritten_question": rewritten}


def retrieve(state: RAGState) -> RAGState:
    query = state.get("rewritten_question") or state["question"]
    docs = retriever.get_relevant_documents(query)
    contents = [d.page_content for d in docs]
    return {"retrieved_docs": contents}


def answer(state: RAGState) -> RAGState:
    context = "\n\n".join(state.get("retrieved_docs", []))
    question = state["question"]

    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a helpful RAG assistant. Use ONLY the context. "
                "If the answer can't be found, say you don't know.",
            ),
            (
```

```
                "human",
                "Context:\n{context}\n\nQuestion: {question}\n\nAnswer:",
            ),
        ]
    )

    chain = prompt | llm | StrOutputParser()
    rag_answer = chain.invoke({"context": context, "question": question})
    return {"answer": rag_answer}
```

4.5 Building the Graph

Now connect the nodes using a `StateGraph`:

```
def build_graph():
    workflow = StateGraph(RAGState)

    workflow.add_node("start", start)
    workflow.add_node("rewrite_query", rewrite_query)
    workflow.add_node("retrieve", retrieve)
    workflow.add_node("answer", answer)

    workflow.set_entry_point("start")

    workflow.add_edge("start", "rewrite_query")
    workflow.add_edge("rewrite_query", "retrieve")
    workflow.add_edge("retrieve", "answer")
    workflow.set_finish_point("answer")

    app = workflow.compile()
    return app
```

4.6 Running the Graph

You can now invoke the compiled app just like an LCEL chain:

```
# src/phase2/run_basic_graph.py
from basic_graph import build_graph

if __name__ == "__main__":
    app = build_graph()
    state = app.invoke({"question": "What is LangGraph?"})
    print("Answer:\n", state["answer"])

    # You can also stream state updates:
    print("\nStreaming state transitions:\n")
    for step in app.stream({"question": "Explain RAG in this course."}):
        print(step)
```

The `.stream` method yields intermediate state snapshots as nodes run, which is useful for debugging.

Module 5 - Control Flow, Agents, Human-in-the-loop & Persistence

This module shows how to add branching, loops, agent-style behavior, human approval, and persistence to your LangGraph workflows.

5.1 Conditional Edges & Branching

You can branch based on state fields using **conditional edges**. For example, if retrieval returns too few documents, you can branch to a re-search node.

```
from typing import Literal
from langgraph.graph import END


def analyze_retrieval(state: RAGState) -> RAGState:
    docs = state.get("retrieved_docs", [])
    if len(docs) < 2:
        return {"error": "Low retrieval confidence: not enough docs."}
    return {}


def decide_next(state: RAGState) -> Literal["re_search", "answer"]:
    # Router node: based on state, choose next step
    if state.get("error"):
        return "re_search"
```

```
    return "answer"


def re_search(state: RAGState) -> RAGState:
    # Could refine query, expand search scope, etc.
    query = state.get("rewritten_question") or state["question"]
    more_docs = retriever.get_relevant_documents(query + " more details")
    contents = state.get("retrieved_docs", []) + [d.page_content for d in more_docs]
    return {"retrieved_docs": contents, "error": ""}

def build_branching_graph():
    workflow = StateGraph(RAGState)

    workflow.add_node("start", start)
    workflow.add_node("rewrite_query", rewrite_query)
    workflow.add_node("retrieve", retrieve)
    workflow.add_node("analyze_retrieval", analyze_retrieval)
    workflow.add_node("router", decide_next)
    workflow.add_node("re_search", re_search)
    workflow.add_node("answer", answer)

    workflow.set_entry_point("start")
    workflow.add_edge("start", "rewrite_query")
    workflow.add_edge("rewrite_query", "retrieve")
    workflow.add_edge("retrieve", "analyze_retrieval")
    workflow.add_edge("analyze_retrieval", "router")

    workflow.add_conditional_edges(
        "router",
        decide_next,
        # If decide_next returns "re_search", go to re_search
        {
            "re_search": "re_search",
            "answer": "answer",
        },
    )

    # After re_search, go to answer and then finish
    workflow.add_edge("re_search", "answer")
    workflow.set_finish_point("answer")

    return workflow.compile()
```

5.2 Agents with LangGraph

A **simple workflow graph** has a fixed sequence of nodes. An **agent graph** contains at least one node where an LLM decides what to do next (e.g. which tool to call, or which edge to take) based on the current state.

You can reuse a LangChain agent (from Phase 1) inside a LangGraph node. The node calls the agent executor and writes the outcome into state:

```
# src/phase2/agent_graph.py
from typing import TypedDict, List

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate
from langgraph.graph import StateGraph


class AgentState(TypedDict, total=False):
    user_input: str
    agent_response: str
    intermediate_steps: List[str]


@tool
def echo_tool(text: str) -> str:
    """Echo the given text (example tool)."""
    return f"Echo: {text}"


def build_tool_agent() -> AgentExecutor:
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    tools = [echo_tool]
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are an assistant that may call tools."),
```

```
            ("human", "{input}"),
        ]
    )
    agent = create_tool_calling_agent(llm, tools, prompt)
    return AgentExecutor(agent=agent, tools=tools, verbose=False)


agent_executor = build_tool_agent()


def agent_node(state: AgentState) -> AgentState:
    """Node that delegates decision-making to a LangChain agent."""
    result = agent_executor.invoke({"input": state["user_input"]})
    # `result` may contain `output` and `intermediate_steps` depending on the agent type
    response = result.get("output", "")
    steps = [str(step) for step in result.get("intermediate_steps", [])]
    return {"agent_response": response, "intermediate_steps": steps}


def build_agent_graph():
    workflow = StateGraph(AgentState)
    workflow.add_node("agent", agent_node)
    workflow.set_entry_point("agent")
    workflow.set_finish_point("agent")
    return workflow.compile()
```

In a more complex agent graph, the agent node could inspect tools' results in `state` and choose which edge to follow next (similar to the router pattern in 5.1, but driven by LLM output).

5.3 Loops & Retries

You can create loops by adding edges that point back to earlier nodes. For example, attempt retrieval up to N times before giving up.

```
class LoopState(RAGState, total=False):
    attempts: int


MAX_ATTEMPTS = 3


def increment_attempts(state: LoopState) -> LoopState:
    attempts = state.get("attempts", 0) + 1
    return {"attempts": attempts}


def decide_retry(state: LoopState) -> str:
    if len(state.get("retrieved_docs", [])) < 1 and state.get("attempts", 0) <
MAX_ATTEMPTS:
        return "retry"
    return "continue"

from langgraph.graph import StateGraph


def build_loop_graph():
    workflow = StateGraph(LoopState)

    workflow.add_node("start", start)
    workflow.add_node("rewrite_query", rewrite_query)
    workflow.add_node("retrieve", retrieve)
    workflow.add_node("increment_attempts", increment_attempts)
    workflow.add_node("decide_retry", decide_retry)
    workflow.add_node("answer", answer)

    workflow.set_entry_point("start")
    workflow.add_edge("start", "rewrite_query")
    workflow.add_edge("rewrite_query", "increment_attempts")
    workflow.add_edge("increment_attempts", "retrieve")
    workflow.add_edge("retrieve", "decide_retry")

    workflow.add_conditional_edges(
        "decide_retry",
        decide_retry,
        {
            "retry": "increment_attempts",
            "continue": "answer",
        },
    )
```

```
    workflow.set_finish_point("answer")
    return workflow.compile()
```

This pattern is useful for robust agents that can retry failed steps or refine queries.

5.4 Human in the Loop & Checkpoints

One of LangGraph's strengths is supporting **pauses** and **checkpoints** so humans can review or approve actions (e.g. sending an email or making a trade).

Conceptually:

1. Graph runs until a node decides to pause.
2. State is persisted at a checkpoint.
3. A human reviews the state and resumes the graph from that point.

A minimal pattern looks like this (actual pause/resume wiring depends on the runtime you use):

```
from langgraph.graph import StateGraph, END

class ApprovalState(TypedDict, total=False):
    draft_answer: str
    approved: bool
    question: str


def draft_answer(state: ApprovalState) -> ApprovalState:
    # Use an LLM to create a draft (similar to earlier answer node)
    # For brevity, imagine we have llm and prompt already defined
    draft = "Draft answer using context and question..."
    return {"draft_answer": draft, "approved": False}


def require_human_approval(state: ApprovalState) -> ApprovalState:
    # In a real app, this would pause and wait for human input.
    # Here we simulate requiring approval by leaving `approved` as False.
    return {}


def send_final_answer(state: ApprovalState) -> ApprovalState:
    if not state.get("approved"):
        return {"draft_answer": state["draft_answer"] + "\n\n(NOT APPROVED YET)"}
    # Send via email / API / UI, etc.
    return state
```

You would then wire these nodes into a graph, and use LangGraph's checkpointing integration to persist `ApprovalState` in a database. When a human approves, you set `approved=True` and resume the graph from the saved checkpoint.

5.5 Persistence & Long-running Sessions

For serious applications, you don't want to lose state when the process exits. LangGraph supports pluggable checkpointers (e.g. in memory, file, or database).

The exact APIs evolve over time, but the high-level pattern is:

- Configure a checkpointer when compiling your graph.
- Each run of the graph is associated with a *thread id* or similar session id.
- When you resume, you pass the same id and the graph continues from the last checkpoint.

This ebook focuses on concepts and basic usage patterns. When you move to production, consult the latest LangGraph docs for the preferred persistence and checkpointing APIs.

Phase 3 - Applications, Deployment & Advanced

In this phase you assemble everything you have learned into **production-style applications**. You will:

- Build a robust RAG assistant orchestrated by LangGraph.
- Design multi-agent systems with clear roles and shared state.
- Learn configuration, testing, deployment, monitoring, and security basics.
- Explore advanced topics and directions for deeper study.

Module 6 - Building a Production-style RAG Assistant with LangGraph

This module combines LangChain (for RAG) with LangGraph (for orchestration) and wraps the result in a simple API server.

6.1 Architecture Overview

At a high level, your system will have:

- **Ingestion pipeline** - load, split, embed, and index documents.
- **RAG graph** - query refinement, retrieval, answer generation.
- **API server** - FastAPI endpoint that calls the graph.
- **Logging & evaluation** - record interactions and test queries.

6.2 Ingestion Script

Create an ingestion script that can be rerun whenever your content changes:

```
# src/phase3/ingest_corpus.py
import os
from pathlib import Path

from dotenv import load_dotenv
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS


DATA_DIR = Path("corpus")
INDEX_DIR = Path("indexes")
INDEX_DIR.mkdir(exist_ok=True)


def load_documents():
    docs = []
    for path in DATA_DIR.glob("**/*.md"):
        loader = TextLoader(str(path))
        docs.extend(loader.load())
    return docs


def main():
    load_dotenv()
    docs = load_documents()
    print(f"Loaded {len(docs)} documents.")
```

```
    splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=150)
    chunks = splitter.split_documents(docs)
    print(f"Split into {len(chunks)} chunks.")

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = FAISS.from_documents(chunks, embedding=embeddings)

    faiss_path = INDEX_DIR / "course_index.faiss"
    index_meta_path = INDEX_DIR / "course_index.pkl"

    vectorstore.save_local(folder_path=str(INDEX_DIR), index_name="course_index")
    print(f"Saved FAISS index to {INDEX_DIR}")


if __name__ == "__main__":
    main()
```

6.3 RAG Graph Orchestration

Next, create a LangGraph graph that encapsulates the query flow. We'll reuse the same index and RAG logic, but introduce nodes for logging and error handling.

```
# src/phase3/rag_graph.py
from typing import TypedDict, List, Optional
from datetime import datetime

from dotenv import load_dotenv

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, END


class RAGSessionState(TypedDict, total=False):
    question: str
    refined_question: str
    retrieved_docs: List[str]
    answer: str
    error: str
    log: List[str]


def build_vectorstore():
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    vectorstore = FAISS.load_local(
        folder_path="indexes",
        index_name="course_index",
        embeddings=embeddings,
        allow_dangerous_deserialization=True,  # local, trusted environment
    )
    return vectorstore.as_retriever(search_kwargs={"k": 4})


def init_state(state: RAGSessionState) -> RAGSessionState:
    log_entry = f"[{datetime.utcnow().isoformat()}] Received question."
    log = state.get("log", [])
    log.append(log_entry)
    return {"log": log}


def refine_question(state: RAGSessionState, llm: ChatOpenAI) -> RAGSessionState:
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "Rewrite the question to be specific and self-contained."),
            ("human", "{question}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    refined = chain.invoke({"question": state["question"]})
    log = state.get("log", [])
    log.append(f"Refined question: {refined}")
    return {"refined_question": refined, "log": log}


def retrieve(state: RAGSessionState, retriever) -> RAGSessionState:
    query = state.get("refined_question") or state["question"]
```

```
    docs = retriever.get_relevant_documents(query)
    contents = [d.page_content for d in docs]
    log = state.get("log", [])
    log.append(f"Retrieved {len(contents)} docs.")
    return {"retrieved_docs": contents, "log": log}


def generate_answer(state: RAGSessionState, llm: ChatOpenAI) -> RAGSessionState:
    context = "\n\n".join(state.get("retrieved_docs", []))
    if not context.strip():
        return {"error": "No relevant context found.", "answer": ""}

    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a helpful assistant for this course. Use only the given
context.",
            ),
            ("human", "Context:\n{context}\n\nQuestion: {question}\n\nAnswer:"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    answer = chain.invoke({"context": context, "question": state["question"]})
    log = state.get("log", [])
    log.append("Generated answer.")
    return {"answer": answer, "log": log}


def build_rag_app():
    load_dotenv()
    retriever = build_vectorstore()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

    workflow = StateGraph(RAGSessionState)

    workflow.add_node("init", lambda s: init_state(s))
    workflow.add_node("refine", lambda s: refine_question(s, llm))
    workflow.add_node("retrieve", lambda s: retrieve(s, retriever))
    workflow.add_node("answer", lambda s: generate_answer(s, llm))

    workflow.set_entry_point("init")
    workflow.add_edge("init", "refine")
    workflow.add_edge("refine", "retrieve")
    workflow.add_edge("retrieve", "answer")
    workflow.set_finish_point("answer")

    app = workflow.compile()
    return app
```

6.4 Exposing the Graph via FastAPI

Now build a small FastAPI server that calls the graph:

```
# src/phase3/api_server.py
from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.middleware.cors import CORSMiddleware

from rag_graph import build_rag_app


class QuestionRequest(BaseModel):
    question: str


class AnswerResponse(BaseModel):
    answer: str
    log: list[str]
    error: str | None = None


app = FastAPI(title="LangGraph RAG Assistant")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
rag_app = build_rag_app()


@app.post("/ask", response_model=AnswerResponse)
async def ask(req: QuestionRequest):
    state = rag_app.invoke({"question": req.question})
    return AnswerResponse(
        answer=state.get("answer", ""),
        log=state.get("log", []),
        error=state.get("error"),
    )
```

Run the server and test:

```
uvicorn src.phase3.api_server:app --reload
```

6.5 Evaluation Basics

Before deployment, you should evaluate the assistant on a small set of test questions. You can start with a simple script that:

- Loads a list of `(question, expected_notes)`.
- Calls the `/ask` endpoint or `rag_app.invoke` directly.
- Logs answers for manual review and calculates latency.

```
# src/phase3/eval_rag.py
import time
from rag_graph import build_rag_app


TEST_QUERIES = [
    "What is LangChain?",
    "What is LangGraph and when should I use it?",
    "Explain what RAG is in this course.",
]


def main():
    app = build_rag_app()
    for q in TEST_QUERIES:
        start = time.time()
        state = app.invoke({"question": q})
        elapsed = time.time() - start

        print("=" * 60)
        print("Question:", q)
        print("Answer:", state.get("answer", ""))
        print("Error:", state.get("error"))
        print(f"Latency: {elapsed:.2f}s")


    if __name__ == "__main__":
    main()
```

6.6 Streaming & Async

For a better user experience, you often want to **stream tokens** from the model to the client as they are generated. LangGraph apps support streaming just like LCEL chains.

You can expose streaming over HTTP using Server-Sent Events (SSE) in FastAPI:

```
# src/phase3/api_server_streaming.py
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from rag_graph import build_rag_app


app = FastAPI(title="LangGraph RAG Assistant (Streaming)")
rag_app = build_rag_app()


def format_stream(question: str):
    # app.stream yields intermediate state updates as the graph runs
    for step in rag_app.stream({"question": question}):
```

```
        # Each `step` is a dict like {"node_name": state}
        # You can choose what to send; here we stream the latest answer fragment.
        for node_name, node_state in step.items():
            answer = node_state.get("answer")
            if answer:
                yield answer


@app.get("/stream")
def stream_answer(question: str):
    return StreamingResponse(
        format_stream(question),
        media_type="text/plain",
    )
```

For heavier workloads, you can make your nodes `async` and call async-friendly tools or models, then use `async for` over `app.astream()` in an async FastAPI endpoint. The design is similar: the graph encapsulates orchestration, and the web layer simply streams or returns results.

Module 7 - Multi-Agent Systems with LangGraph

Multi-agent systems use multiple specialized agents that collaborate on a task: for example, a planner, a researcher, a writer, and a reviewer.

7.1 Design Patterns

Common patterns include:

- **Planner-worker** - planner breaks down tasks, workers execute them.
- **Specialists** - different agents for search, code, writing, etc.
- **Hierarchy** - a controller agent routes tasks to sub-agents.

7.2 Shared State for Multi-Agent Graphs

All agents operate over a shared state structure. Define it carefully:

```
# src/phase3/multi_agent_graph.py
from typing import TypedDict, List


class MultiAgentState(TypedDict, total=False):
    goal: str
    plan: List[str]
    research_notes: str
    draft_report: str
    reviewed_report: str
    messages: List[str]
```

7.3 Planner Agent Node

The planner takes a goal and produces a list of concrete steps:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


def build_llm():
    return ChatOpenAI(model="gpt-4o-mini", temperature=0.2)


def planner_node(state: MultiAgentState) -> MultiAgentState:
    llm = build_llm()
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a project planner. Break the goal into 3-6 numbered
steps."),
            ("human", "{goal}"),
        ]
```

```
    )
    chain = prompt | llm | StrOutputParser()
    plan_text = chain.invoke({"goal": state["goal"]})

    steps = []
    for line in plan_text.splitlines():
        line = line.strip()
        if not line:
            continue
        # Rough parsing of lines like "1. Do X"
        if "." in line:
            line = line.split(".", 1)[1].strip()
        steps.append(line)

    messages = state.get("messages", [])
    messages.append("Planner produced steps.")
    return {"plan": steps, "messages": messages}
```

7.4 Research Agent Node

The research agent could call tools (web search, RAG) to gather information; here we will simulate with the LLM:

```
def research_node(state: MultiAgentState) -> MultiAgentState:
    llm = build_llm()
    plan = state.get("plan", [])
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a research assistant. For each step, write concise notes.",
            ),
            ("human", "Goal: {goal}\n\nSteps:\n{steps}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    steps_text = "\n".join(f"- {s}" for s in plan)
    notes = chain.invoke({"goal": state["goal"], "steps": steps_text})

    messages = state.get("messages", [])
    messages.append("Researcher produced notes.")
    return {"research_notes": notes, "messages": messages}
```

7.5 Writer & Reviewer Agents

```
def writer_node(state: MultiAgentState) -> MultiAgentState:
    llm = build_llm()
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a technical writer. Draft a clear, structured report."),
            ("human", "Goal:\n{goal}\n\nResearch notes:\n{notes}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    draft = chain.invoke({"goal": state["goal"], "notes": state["research_notes"]})
    messages = state.get("messages", [])
    messages.append("Writer produced draft.")
    return {"draft_report": draft, "messages": messages}


def reviewer_node(state: MultiAgentState) -> MultiAgentState:
    llm = build_llm()
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a strict reviewer. Fix style, structure, and obvious
hallucinations. "
                "If information seems unsupported, flag it.",
            ),
            ("human", "Draft report:\n{draft}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    reviewed = chain.invoke({"draft": state["draft_report"]})
    messages = state.get("messages", [])
    messages.append("Reviewer edited draft.")
```

```
    return {"reviewed_report": reviewed, "messages": messages}
```

7.6 Moderation Node

Add a simple moderation node that checks for dangerous content:

```
def moderation_node(state: MultiAgentState) -> MultiAgentState:
    report = state.get("reviewed_report") or state.get("draft_report", "")
    # Simple heuristic check; in production, integrate with a moderation API.
    banned_keywords = ["password", "credit card", "ssn"]
    flags = [kw for kw in banned_keywords if kw.lower() in report.lower()]

    messages = state.get("messages", [])
    if flags:
        messages.append(f"Moderation warning: found {flags}.")
        safe_report = report + "\n\n[WARNING: Potential sensitive data removed.]"
        return {"reviewed_report": safe_report, "messages": messages}

    messages.append("Moderation passed.")
    return {"messages": messages}
```

7.7 Wiring the Multi-Agent Graph

```
from langgraph.graph import StateGraph


def build_multi_agent_app():
    workflow = StateGraph(MultiAgentState)

    workflow.add_node("planner", planner_node)
    workflow.add_node("researcher", research_node)
    workflow.add_node("writer", writer_node)
    workflow.add_node("reviewer", reviewer_node)
    workflow.add_node("moderation", moderation_node)

    workflow.set_entry_point("planner")
    workflow.add_edge("planner", "researcher")
    workflow.add_edge("researcher", "writer")
    workflow.add_edge("writer", "reviewer")
    workflow.add_edge("reviewer", "moderation")
    workflow.set_finish_point("moderation")

    return workflow.compile()

# src/phase3/run_multi_agent.py
from multi_agent_graph import build_multi_agent_app


def main():
    app = build_multi_agent_app()
    state = app.invoke(
        {"goal": "Write a short report explaining LangChain and LangGraph to a new
engineer."}
    )
    print("Messages:")
    for msg in state.get("messages", []):
        print("-", msg)

    print("\nFinal report:\n")
    print(state.get("reviewed_report") or state.get("draft_report", "No report"))


if __name__ == "__main__":
    main()
```

Module 8 - Hardening & Deploying LangChain + LangGraph Apps

This module covers configuration management, testing, deployment, monitoring, and basic security concerns.

8.1 Configuration & Environment Management

Avoid hard-coding model names, temperatures, and service URLs. Instead, centralize them:

```
# src/phase3/config.py
import os
from dataclasses import dataclass
```

```
from dotenv import load_dotenv


@dataclass
class ModelConfig:
    chat_model: str = "gpt-4o-mini"
    temperature: float = 0.2
    embedding_model: str = "text-embedding-3-small"


@dataclass
class AppConfig:
    environment: str
    model: ModelConfig


def load_config() -> AppConfig:
    load_dotenv()
    env = os.getenv("APP_ENV", "dev")
    model = ModelConfig(
        chat_model=os.getenv("CHAT_MODEL", "gpt-4o-mini"),
        temperature=float(os.getenv("CHAT_TEMPERATURE", "0.2")),
        embedding_model=os.getenv("EMBEDDING_MODEL", "text-embedding-3-small"),
    )
    return AppConfig(environment=env, model=model)
```

8.2 Testing

You should test both individual components and end-to-end flows:

- **Unit tests** - nodes, utility functions, prompt formatters.
- **Golden tests** - fixed input and expected output snapshots.
- **Regression tests** - re-run tests when you change prompts or models.

```
# tests/test_rewrite_query.py
from phase2.basic_graph import rewrite_query, RAGState


def test_rewrite_query_makes_nonempty_string():
    state: RAGState = {"question": "What is LangChain?"}
    new_state = rewrite_query(state)
    assert "rewritten_question" in new_state
    assert isinstance(new_state["rewritten_question"], str)
    assert new_state["rewritten_question"].strip()
```

8.3 Deployment Options

Common deployment patterns include:

- Containerized FastAPI app behind a load balancer.
- Serverless function (for short-lived requests).
- Background workers for long-running agents (with queues).

A minimal Dockerfile might look like:

```
# Dockerfile (sketch)
FROM python:3.11-slim

WORKDIR /app
COPY pyproject.toml poetry.lock ./  # or requirements.txt
RUN pip install --no-cache-dir fastapi uvicorn langchain langgraph ...

COPY src ./src

CMD ["uvicorn", "src.phase3.api_server:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.4 Monitoring & Observability
For real applications, you should:

- Log inputs, outputs, and errors with correlation IDs.
- Track latency and cost per request.
- Use traces (e.g. via LangSmith or similar tools) to inspect model calls and graph steps.

```
import logging
import uuid
from datetime import datetime

logger = logging.getLogger("rag_app")
logging.basicConfig(level=logging.INFO)


def log_request(question: str):
    request_id = str(uuid.uuid4())
    logger.info("request_id=%s question=%s", request_id, question)
    return request_id


def log_response(request_id: str, answer: str, error: str | None):
    logger.info(
        "request_id=%s answer_length=%d error=%s",
        request_id,
        len(answer),
        error,
    )
```

8.5 Security & Guardrails
Key concerns:

- Secure API keys using environment variables or a secret manager.
- Validate and sanitize tool inputs to avoid command injection.
- Limit what tools can access (e.g. file system, network).
- Use moderation (Module 7) to filter harmful content.

Treat any model output as untrusted input. Tools that perform side-effects (sending emails, modifying files, making payments) must have extra validation layers.

Module 9 - Advanced Topics & Next Steps (Optional)
This final module offers directions for deeper specialization. Choose the areas that best fit your goals.

9.1 Advanced Retrieval
Beyond basic vector search:

- **Hybrid search** - combine BM25 (keyword) with dense vectors.
- **Reranking** - use a cross-encoder or LLM to rerank retrieved chunks.
- **Hierarchical indexing** - parent/child documents, multi-vector per document.

In LangChain, you can often swap out the retriever with minimal changes to your RAG chain/graph.

9.2 Domain-Specific Agents

Examples:

- **Code agents** - tools to read/write files, run tests, and edit code.
- **Data analysis agents** - tools over Python, pandas, SQL databases.
- **DevOps agents** - tools to inspect logs, metrics, and configs.

The patterns you learned in Modules 3 and 7 apply: define tools carefully, give the agent a clear role, and use LangGraph to orchestrate multi-step flows.

9.3 Performance & Cost Tuning

Practical tips:

- Use smaller, cheaper models where quality allows; reserve larger models for critical steps.
- Reduce context size by improving retrieval and prompt design.
- Cache embeddings and model outputs where appropriate.
- Run steps in parallel when they don't depend on each other (async and LCEL parallelism).

9.4 Reading Source Code

To deeply understand LangChain and LangGraph, spend time reading selected parts of their source code:

- Core runnable and chain implementations.
- Retrievers and vector store integrations.
- StateGraph and execution engine in LangGraph.

Reading real-world Python code is a powerful way to level up your skills.

9.5 Staying Up to Date

The LLM ecosystem changes fast. To stay current:

- Follow LangChain and LangGraph release notes.
- Study example repositories and community projects.
- Periodically revisit your prompts, tools, and graphs as new models appear.

You now have a full stack: from basic model calls to orchestrated, production-style applications. The next step is to build something real for your own use case and iterate based on real feedback.

Phase 4 - Advanced Agentic Systems

This phase assumes you are comfortable with basic LangChain agents and LangGraph workflows. You now focus on **advanced agentic patterns**: richer reasoning, hierarchical planning, multimodal and code-execution tools, long-term memory, safety & governance, and scalable multi-agent systems.

Module 10 - Advanced Agent Reasoning Patterns

This module moves beyond simple "ask a model, maybe call a tool" agents into richer reasoning patterns such as ReAct, reflexion, and multi-branch reasoning.

10.1 ReAct-style Agents (Reason + Act)

ReAct (Reason + Act) is a pattern where the model explicitly emits:

- a *Thought* - natural language reasoning, and
- an *Action* - which tool to call (with arguments).

A simple ReAct loop can be implemented in LangChain with a prompt that encourages this structure:

```
# src/phase4/react_agent.py
from typing import List, Tuple
import re
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool


@tool
def search_notes(query: str) -> str:
    """Search the course notes for information related to the query (placeholder)."""
    # In a real app, call your RAG pipeline instead.
    return f"[Search results for: {query}]"


TOOLS = {
    "search_notes": search_notes,
}


def parse_react_output(text: str) -> Tuple[str, str, str]:
    """
    Parse a simple ReAct style output of the form:
      Thought: ...
      Action: search_notes["query"]
    """
    thought_match = re.search(r"Thought:(.*)", text, re.DOTALL)
    action_match = re.search(r"Action:\s*(\w+)\[(.*)\]", text)

    thought = thought_match.group(1).strip() if thought_match else ""
    if not action_match:
        return thought, "", ""
    tool_name = action_match.group(1).strip()
    arg = action_match.group(2).strip().strip('"').strip("'")
    return thought, tool_name, arg


def run_react_loop(question: str, max_steps: int = 3) -> str:
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

    history: List[str] = []
    observations: List[str] = []

    for step in range(max_steps):
        prompt = (
            "You are a ReAct agent. Use Thought/Action/Observation steps. "
            "You have a tool: search_notes[query].\n\n"
            f"Question: {question}\n\n"
        )
        if observations:
```

```
            prompt += "Previous observations:\n" + "\n".join(observations) + "\n\n"

        prompt += (
            "Respond strictly in this format:\n"
            "Thought: <your reasoning>\n"
            "Action: tool_name[\"argument\"] OR Action: finish[\"final answer\"]\n"
        )

        response = llm.invoke(prompt)
        text = response.content
        history.append(text)

        thought, tool_name, arg = parse_react_output(text)
        if tool_name == "finish":
            return arg  # final answer

        tool = TOOLS.get(tool_name)
        if not tool:
            observations.append(f"Observation: Unknown tool '{tool_name}'.")
            continue

        tool_result = tool.invoke({"query": arg})
        observations.append(f"Observation: {tool_result}")

    # If loop ended without finish, summarize best effort
    return "I could not fully answer within the step limit. Partial reasoning:\n" +
"\n".join(history)
```

This example shows the core idea: the model emits a structured "Thought" and "Action" block; your code parses it, calls tools, and feeds observations back in the next step.

10.2 Reflexion / Self-Critique Loop

Reflexion patterns add a meta-step where the agent critiques its own answers and revises them. You can implement this with a follow-up prompt that asks the model to evaluate and refine its own output.

```
# src/phase4/reflexion.py
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate


def answer_and_reflect(question: str) -> str:
    load_dotenv()
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.5)

    # Step 1: initial answer
    base_prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a helpful assistant."),
            ("human", "{question}"),
        ]
    )
    base_chain = base_prompt | llm | StrOutputParser()
    draft = base_chain.invoke({"question": question})

    # Step 2: critique and revise
    reflexion_prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a strict reviewer. Check the answer for correctness, missing
details, "
                "and hallucinations. Then rewrite a final, improved answer.",
            ),
            (
                "human",
                "Question:\n{question}\n\nDraft answer:\n{draft}\n\n"
                "First, list potential issues. Then provide a corrected final answer.",
            ),
        ]
    )
    reflexion_chain = reflexion_prompt | llm | StrOutputParser()
    final_answer = reflexion_chain.invoke({"question": question, "draft": draft})
    return final_answer
```

10.3 Multi-Branch / Tree-of-Thought-style Reasoning

In more complex tasks, you may want the model to generate multiple candidate solutions ("branches"), score them, and pick the best. A light-weight approach:

1. Ask the model to generate N reasoning paths.
2. Ask it (or another model) to score each path.
3. Pick the highest-scoring final answer.

```
# src/phase4/multi_branch.py
from typing import List
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


def generate_branches(question: str, n: int = 3) -> List[str]:
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "Generate multiple different reasoning paths to solve the problem.",
            ),
            (
                "human",
                "Question: {question}\n\nGenerate {n} distinct candidate answers with
detailed reasoning, "
                "labeled as 'Candidate 1', 'Candidate 2', etc.",
            ),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    text = chain.invoke({"question": question, "n": n})
    # For teaching purposes, we return the raw response; in production, parse each
candidate explicitly.
    return [text]


def score_and_select(question: str, candidates: List[str]) -> str:
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
    scoring_prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are a grader. Evaluate candidate answers and pick the best one.",
            ),
            (
                "human",
                "Question:\n{question}\n\nCandidates:\n{candidates}\n\n"
                "Choose the best candidate and explain briefly why.",
            ),
        ]
    )
    chain = scoring_prompt | llm | StrOutputParser()
    return chain.invoke({"question": question, "candidates": "\n\n".join(candidates)})
```

Module 11 - Planning & Hierarchical Agents

Planning agents separate *planning* (deciding what to do) from *execution* (doing it). This structure makes complex tasks more reliable and inspectable.

11.1 Plan-and-Execute Pattern

A Plan-and-Execute agent typically involves:

- A **planner** that converts a high-level goal into a list of steps.
- An **executor** that executes steps using tools, RAG, or

other agents.

```
# src/phase4/plan_and_execute.py
from typing import List
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


def build_llm():
    return ChatOpenAI(model="gpt-4o-mini", temperature=0.2)


def plan_steps(goal: str) -> List[str]:
    llm = build_llm()
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a planner. Break the goal into 3-7 concrete steps."),
            ("human", "{goal}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    raw_plan = chain.invoke({"goal": goal})
    steps: List[str] = []
    for line in raw_plan.splitlines():
        line = line.strip()
        if not line:
            continue
        if "." in line:
            line = line.split(".", 1)[1].strip()
        steps.append(line)
    return steps


def execute_step(step: str) -> str:
    llm = build_llm()
    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are an executor. Perform the step and report results
clearly."),
            ("human", "Step: {step}"),
        ]
    )
    chain = prompt | llm | StrOutputParser()
    return chain.invoke({"step": step})


def plan_and_execute(goal: str) -> str:
    load_dotenv()
    steps = plan_steps(goal)
    reports = []
    for i, step in enumerate(steps, start=1):
        result = execute_step(step)
        reports.append(f"Step {i}: {step}\nResult:\n{result}")
    return "\n\n".join(reports)
```

11.2 Wrapping Planner & Executor in LangGraph

You can wrap this pattern in a LangGraph to track state and allow dynamic replanning:

```
# src/phase4/plan_graph.py
from typing import TypedDict, List
from langgraph.graph import StateGraph
from plan_and_execute import plan_steps, execute_step


class PlanState(TypedDict, total=False):
    goal: str
    steps: List[str]
    current_index: int
    reports: List[str]


def node_plan(state: PlanState) -> PlanState:
    steps = plan_steps(state["goal"])
    return {"steps": steps, "current_index": 0, "reports": []}
```

```
def node_execute(state: PlanState) -> PlanState:
    idx = state.get("current_index", 0)
    steps = state["steps"]
    if idx >= len(steps):
        return {}
    step = steps[idx]
    result = execute_step(step)
    reports = state.get("reports", [])
    reports.append(f"{step}\nResult:\n{result}")
    return {"reports": reports, "current_index": idx + 1}


def node_decide_next(state: PlanState) -> str:
    if state.get("current_index", 0) < len(state.get("steps", [])):
        return "execute"
    return "finish"


def build_plan_graph():
    workflow = StateGraph(PlanState)
    workflow.add_node("plan", node_plan)
    workflow.add_node("execute", node_execute)
    workflow.add_node("decider", node_decide_next)

    workflow.set_entry_point("plan")
    workflow.add_edge("plan", "execute")
    workflow.add_edge("execute", "decider")

    workflow.add_conditional_edges(
        "decider",
        node_decide_next,
        {
            "execute": "execute",
            "finish": "__end__",  # special END label; actual API may differ
        },
    )

    return workflow.compile()
```

Check the latest LangGraph docs for the exact constant representing the end node (often `END`). The pattern above is what matters: a planner node, an executor node, and a decision node that loops.

Module 12 - Advanced Tools, Multimodal & Code Agents

In this module you extend agents with richer tools: multiple tool pipelines, multimodal tools, and code/DB execution tools.

12.1 Dynamic Toolsets & Tool Pipelines

As your application grows, you may have many tools. You can dynamically choose which tools to expose based on user, environment, or state.

```
# src/phase4/dynamic_tools.py
from typing import List
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate


@tool
def web_search(query: str) -> str:
    """Search the web for the given query (placeholder)."""
    return f"[Web search results for: {query}]"


@tool
def summarize_text(text: str) -> str:
    """Summarize a long piece of text."""
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return llm.invoke(f"Summarize this:\n\n{text}").content


def build_agent_for_mode(mode: str) -> AgentExecutor:
    load_dotenv()
```

```
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

    base_prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a versatile assistant that can use tools."),
            ("human", "{input}"),
        ]
    )

    if mode == "research":
        tools = [web_search, summarize_text]
    else:
        tools = [summarize_text]

    agent = create_tool_calling_agent(llm, tools, base_prompt)
    return AgentExecutor(agent=agent, tools=tools, verbose=True)
```

12.2 Multimodal Tools (Images, Documents)

Many modern models can see images and PDFs. In LangChain, you typically:

- Implement a tool that accepts an image path or bytes.
- Use a multimodal-capable model under the hood.

A sketch of an "analyze image" tool (pseudo-code, since exact API depends on the provider):

```
# src/phase4/image_tool.py
from langchain_core.tools import tool


@tool
def analyze_image(path: str) -> str:
    """
    Analyze an image at the given path and describe it.
    In practice, use a multimodal model (e.g. OpenAI Vision) here.
    """
    # Example (pseudo-code, adjust to real API):
    # from langchain_openai import ChatOpenAI
    # llm = ChatOpenAI(model="gpt-4o-mini")  # vision-capable variant
    # with open(path, "rb") as f:
    #     img_bytes = f.read()
    # response = llm.invoke([ImageMessage(img_bytes), TextMessage("Describe this
image.")])
    # return response.content
    return f"(Stub) Would analyze image at {path} using a vision model."
```

12.3 Code-Interpreter / Python REPL Tools

Code-execution tools are powerful but must be sandboxed. Here is a minimal, constrained interpreter:

```
# src/phase4/code_interpreter.py
from typing import Any, Dict
from langchain_core.tools import tool


SAFE_GLOBALS: Dict[str, Any] = {
    "__builtins__": {
        "abs": abs,
        "min": min,
        "max": max,
        "sum": sum,
        "len": len,
        "range": range,
    }
}


@tool
def python_eval(code: str) -> str:
    """
    Evaluate a small Python expression safely (no imports, no IO).
    Intended for numeric or simple list/dict operations.
    """
    try:
        result = eval(code, SAFE_GLOBALS, {})
        return repr(result)
```

```
    except Exception as e:
        return f"Error: {e}"
```

You can expose this tool to an agent with a clear system prompt that restricts what kind of code it is allowed to generate (no file/network access, no imports).

Module 13 - Memory, Safety, Governance & Evaluation

This module strengthens your agents with long-term memory, explicit safety rules, and evaluation practices.

13.1 Long-Term Memory with a Vector Store

Instead of only relying on conversation buffers, you can store key events into a vector store and let the agent retrieve them later as "memory".

```
# src/phase4/vector_memory.py
from typing import List
from dataclasses import dataclass

from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


@dataclass
class MemoryStore:
    vectorstore: FAISS

    @classmethod
    def create(cls):
        embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
        # Start empty
        return cls(vectorstore=FAISS.from_texts([], embeddings))

    def add_event(self, text: str, metadata: dict | None = None):
        self.vectorstore.add_texts([text], metadatas=[metadata or {}])

    def recall(self, query: str, k: int = 5) -> List[str]:
        docs = self.vectorstore.similarity_search(query, k=k)
        return [d.page_content for d in docs]


def chat_with_memory(question: str, memory: MemoryStore) -> str:
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "You are an assistant with long-term memory. Use the provided memory
snippets if helpful.",
            ),
            (
                "human",
                "Memory snippets:\n{memories}\n\nUser question:\n{question}",
            ),
        ]
    )
    chain = prompt | llm | StrOutputParser()

    memories = memory.recall(question)
    memories_text = "\n\n".join(memories) if memories else "(no relevant memories)"
    answer = chain.invoke({"memories": memories_text, "question": question})

    # After answering, store this exchange as a new memory
    memory.add_event(f"Q: {question}\nA: {answer}", metadata={"type": "qa"})
    return answer
```

13.2 Safety Policies Around Tool Use

You can implement a simple policy engine that decides whether a tool call is allowed. For example, forbid certain dangerous patterns or domains.

```
# src/phase4/tool_policy.py
from typing import Dict, Any
```

```
def is_tool_call_allowed(tool_name: str, args: Dict[str, Any]) -> bool:
    # Example rules:
    if tool_name == "python_eval":
        code = args.get("code", "")
        if "import" in code or "__" in code:
            return False
    if tool_name == "web_search":
        query = args.get("query", "")
        if "password" in query.lower():
            return False
    return True


def guard_tool_call(tool_name: str, args: Dict[str, Any], call_fn):
    if not is_tool_call_allowed(tool_name, args):
        return f"Policy blocked tool '{tool_name}' with args {args}"
    return call_fn(**args)
```

13.3 Logging & Audit Trails

For governance and debugging, log every tool call and decision:

```
import logging
import uuid

logger = logging.getLogger("agent_audit")
logging.basicConfig(level=logging.INFO)


def log_tool_call(tool_name: str, args: dict, result: str, allowed: bool, run_id: str |
None = None):
    if run_id is None:
        run_id = str(uuid.uuid4())
    logger.info(
        "run_id=%s tool=%s allowed=%s args=%s result_snippet=%s",
        run_id,
        tool_name,
        allowed,
        args,
        result[:120].replace("\n", " "),
    )
```

13.4 Evaluating Agent Behavior

You can build simple evaluation harnesses around your agents:

- Define a dataset of tasks with expected behaviors.
- Run your agents on each task; log outputs and metrics (success/failure, latency, cost).
- Use automatic checks plus manual review.

For deeper evaluation, you can use tools like LangSmith to capture traces and run evaluators on top.

Module 14 - Multi-Agent Coordination & Scalable Deployment

This final module looks at multi-agent collaboration patterns and what it takes to run many agent sessions in production.

14.1 Debate & Critique Agents

A simple debate pattern:

1. Two agents independently produce answers.
2. A third "judge" agent compares and chooses.

```
# src/phase4/debate.py
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser


def build_debater(name: str) -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0.7)


def debate(question: str) -> str:
    load_dotenv()
    llm1 = build_debater("Agent A")
    llm2 = build_debater("Agent B")

    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are {name}. Provide a detailed answer."),
            ("human", "{question}"),
        ]
    )
    chain = prompt | StrOutputParser()

    a_answer = (prompt | llm1 | StrOutputParser()).invoke({"name": "Agent A", "question":
question})
    b_answer = (prompt | llm2 | StrOutputParser()).invoke({"name": "Agent B", "question":
question})

    judge_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)
    judge_prompt = ChatPromptTemplate.from_messages(
        [
            ("system", "You are a judge. Compare two answers and pick the better one."),
            (
                "human",
                "Question:\n{question}\n\nAnswer A:\n{a}\n\nAnswer B:\n{b}\n\n"
                "Explain which answer is better and why. Then restate the final answer.",
            ),
        ]
    )
    judge_chain = judge_prompt | judge_llm | StrOutputParser()
    return judge_chain.invoke({"question": question, "a": a_answer, "b": b_answer})
```

14.2 Multi-Agent LangGraph Patterns

In LangGraph, you can represent each agent as a node or subgraph. A debate graph might:

- Have nodes for `agent_a`, `agent_b`, and `judge`.
- Store their answers in shared state.
- End with a `final_answer` field chosen by the judge.

14.3 Scaling Agent Sessions

For production workloads:

- Run agents in containers (e.g. FastAPI + LangGraph inside Docker).
- Use a queue or event system to manage long-running tasks.
- Implement backpressure and rate limiting against LLM APIs.
- Isolate user data by using per-user state keys and separate memory stores.

Many of these patterns are architecture-specific, but the mental model stays the same: LangChain handles modeling and tools, LangGraph orchestrates multi-step and multi-agent flows, and your infrastructure provides scaling, queues, and storage.